WEB TECHNOLOGIES

Unit 2: Web Systems Architecture & Scalability

Rgpvnotes.tech - Your Academic Excellence Partner

UNIT 2: Web Systems Architecture | Client/Server (2-Tier) Architecture | 3-Tier Architecture | Fast & Scalable Data Access | Caches | Proxies | Indexes | Load Balancers | Queues | Web Application Architecture (WAA)

Table of Contents

Topic 1: Overview of Web Systems Architecture

Topic 2: Client/Server (2-Tier) Architecture

Topic 3: 3-Tier Architecture

Topic 4: Caches (Fast Data Access)

Topic 5: Proxies and Indexes

Topic 6: Load Balancers and Queues

Topic 7: Web Application Architecture (WAA)

Revision Notes

Important Questions

1 Overview of Web Systems Architecture

Definition:

Web Systems Architecture is the conceptual layout and structural design of a web-based system. It defines how various components (like clients, servers, databases, and networks) interact, communicate, and work together to deliver web services efficiently and securely.

Detailed Explanation:

Building a website for 10 users is vastly different from building a web system for 10 million users. Web

Systems Architecture is the blueprint that dictates how an application will scale, handle errors, process data, and serve users without crashing. It represents the high-level structural decisions that are very difficult to change once the system is built.

A good web architecture separates the application into logical layers (Tiers). This separation of concerns means that if you want to upgrade your database, you don't have to rewrite your user interface. The primary goal of web architecture is to fulfill both **Functional Requirements** (what the system should do, e.g., process payments) and **Non-Functional Requirements** (how well the system should do it, e.g., load in under 2 seconds, support 50,000 concurrent users).

Modern web architecture focuses heavily on **Scalability** (the ability to handle growing amounts of work by adding resources) and **High Availability** (ensuring the system is almost always online, even if some parts fail).

🔍 Key Characteristics of Good Architecture:

- ✓ **Scalability:** System can handle increased load seamlessly (Vertical or Horizontal scaling).
- ✓ **Reliability & Availability:** System remains functional despite hardware or software failures (Fault Tolerance).
- ✓ **Security:** Protects data from unauthorized access at every layer.
- ✓ **Maintainability:** Code and infrastructure are organized so developers can update features easily.
- ✓ **Performance:** Optimized data flow for fast response times.

🌍 Applications:

→ Designing streaming platforms like Netflix to deliver video globally without buffering.

→ Architecting banking systems to ensure transaction security and zero data loss.

→ Building social media networks to handle millions of simultaneous user interactions.

👉 ***Hinglish Explanation:***

Web Systems Architecture ek blueprint (naksha) ki tarah hai jo batata hai ki website ke alag-alag parts (jaise frontend, backend, database) aapas mein kaise connect honge. Ek achhi architecture ka main kaam yeh ensure karna hai ki agar website par achanak se bahut saare users aa jayein, toh website crash na ho aur fast chalti rahe.

2 Client/Server (2-Tier) Architecture

📌 Definition:

2-Tier Architecture (also known as standard Client/Server architecture) is a software architecture pattern

where the presentation layer (Client) runs on a user's device and communicates directly with the data storage layer (Server) without any intermediary middleware.

📖 Detailed Explanation:

In a 2-Tier architecture, the system is strictly divided into two parts: the Client and the Server. The **Client Tier** contains the User Interface (UI) and the application's core business logic. It runs on the user's local machine (like a desktop application or a rich client). The **Server Tier** is responsible exclusively for database management and data storage.

When the client needs data, it establishes a direct connection (using protocols like ODBC or JDBC) to the database server. The client requests data, the server processes the query and returns the results, and the client then formats and displays that data to the user. Because the business logic resides on the client machine, these are often referred to as "Fat Clients."

While this architecture is extremely fast for small networks, it has major limitations for web applications. If the business logic needs an update, the application must be updated on every single client machine. Furthermore, opening direct database connections to thousands of clients over the internet is a massive security risk and causes the database server to quickly run out of connection resources.

🔑 Key Features:

- ✓ **Direct Communication:** Client talks directly to the database.
- ✓ **Fat Client:** Business logic is embedded in the client application.
- ✓ **High Performance (Locally):** Very fast data retrieval on Local Area Networks (LANs).
- ✓ **Limited Connections:** Databases can only handle a limited number of direct, persistent connections.

📊 2-Tier Architecture Diagram

```
     [ TIER 1: CLIENT ]                [ TIER 2: SERVER ]
+----------------------+           +----------------------+
|                      |           |                      |
|  User Interface (UI) |   Direct  |   Database Engine    |
|          +           | <=======> |          +           |
|    Business Logic    |   Query   |     Data Storage     |
|    ("Fat Client")    |           |                      |
+----------------------+           +----------------------+
```

✅ Advantages:

- ✓ Very easy and quick to develop and setup.
- ✓ Excellent performance for small user groups on a local network.
- ✓ System structure is simple to understand and troubleshoot.

❌ Disadvantages:

- ✗ **Poor Scalability:** Cannot handle a large number of concurrent users.
- ✗ **Security Risks:** Database is directly exposed to the client.

✗ **Maintenance Nightmare:** Business logic updates require redeploying the app to all clients.

Real-Life Example:

A traditional desktop billing software installed in a local supermarket. The software on the cashier's computer (Client) directly connects to a MySQL database installed on a back-office computer (Server) to retrieve product prices and save invoices.

Hinglish Explanation:

2-Tier architecture mein sirf do layers hoti hain: Client (jaise aapka computer) aur Server (Database). Client direct database se baat karta hai. Isme saara main dimaag (business logic) client software ke andar hota hai. Ye chhote systems ke liye bahut fast hai, par badi websites ke liye fail ho jata hai kyunki database direct internet pe expose karna safe nahi hai aur hazaron log ek sath direct connect nahi ho sakte.

3 3-Tier Architecture

Definition:

3-Tier Architecture is a software design pattern that divides an application into three logical and physical computing tiers: the Presentation Tier (User Interface), the Application Tier (Business Logic), and the Data Tier (Database).

Detailed Explanation:

To overcome the limitations of the 2-Tier model, the 3-Tier architecture introduces a "middleman." It isolates the application into three distinct layers, ensuring that the client never communicates directly with the database. This separation of concerns allows each tier to be developed, updated, and scaled independently.

1. Presentation Tier (Client Layer): This is the topmost level, represented by a Web Browser or Mobile App. Its only job is to display information to the user and collect user inputs. It contains HTML, CSS, and UI-focused JavaScript. It is a "Thin Client."

2. Application Tier (Business Logic / Middle Tier): This is the brain of the application. Hosted on a Web Server or Application Server (using languages like Java, Python, Node.js, PHP), it processes the user's inputs, makes logical decisions, performs calculations, and decides what data is needed.

3. Data Tier (Database Layer): This tier consists of Database Management Systems (like MySQL, PostgreSQL, MongoDB). It stores and retrieves data based strictly on the requests it receives from the Application Tier.

Because the Application Tier acts as a shield, the Database Tier is protected from direct external access, drastically improving security. Furthermore, if the app gets popular, you can add more Application servers (horizontal scaling) without touching the Client or Data tiers.

🔑 Key Features:

- ✓ **Separation of Concerns:** UI, logic, and data are completely isolated.
- ✓ **Thin Client:** The client device only handles rendering, not processing.
- ✓ **Enhanced Security:** The database is hidden behind the application server firewall.
- ✓ **High Scalability:** Individual tiers can be scaled on separate hardware servers as needed.

📊 3-Tier Architecture Diagram

```
  [ TIER 1: PRESENTATION ]      [ TIER 2: APPLICATION ]       [ TIER 3: DATA ]
+-------------------------+    +-----------------------+    +------------------+
|    Web Browser / App    |    |   Application Server  |    |  Database Server |
|      (HTML/CSS/JS)      |    |    (Python/Java/PHP)  |    |  (MySQL/Oracle)  |
|                         |    |                       |    |                  |
|  Displays UI & Catches  |<==>| Processes Rules, Does |<==>| Stores & Fetches |
|     User Inputs         |HTTP| Calculations & Checks |SQL | Raw Data Securely|
+-------------------------+    +-----------------------+    +------------------+
```

✅ Advantages:

- ✓ **Highly Secure:** Clients cannot access the database directly.
- ✓ **Maintainable:** Changing the business logic doesn't require updating the client's browser/app.
- ✓ **Scalable:** You can upgrade the Application server independently if traffic spikes.

❌ Disadvantages:

- ✗ **Increased Complexity:** Harder to design and set up compared to 2-Tier.
- ✗ **Network Latency:** Data has to travel through an extra hop (Client -> App -> DB), which can introduce slight delays.

💡 Real-Life Example:

An online banking website.
Tier 1: The website on your phone showing your balance (Presentation).
Tier 2: The bank's server verifying your password and checking if you have enough funds to make a transfer (Application).
Tier 3: The secure database deep in the bank's data center that actually deducts the money from your account record (Data).

👉 *Hinglish Explanation:*

*3-Tier architecture mein system ko 3 parts mein baant dete hain: Pehla **Presentation** (jo user ko dikhta hai), doosra **Application** (jo saara dimaag lagata hai aur calculations karta hai), aur teesra **Data** (jahan files save hoti hain). Isme client direct database se connect nahi hota, balki middle layer (Application) ke through request bhejta hai. Ye badi websites ke liye best hai kyunki ye highly secure aur easily upgrade (scale) hone wali architecture hai.*

4 Building Blocks for Fast Access: Caches

📌 Definition:

A **Cache** is a high-speed data storage layer (usually stored in RAM) that temporarily stores a subset of data so that future requests for that data are served much faster than accessing the primary storage location (like a hard drive or database).

📖 Detailed Explanation:

In web architecture, databases are usually the biggest bottleneck. Reading data from a hard drive (where databases live) is slow compared to reading from RAM. When thousands of users request the exact same data (e.g., the top 10 trending movies on Netflix), querying the database thousands of times is highly inefficient and can crash the system.

This is where **Caching** comes in. When a piece of data is requested for the first time, the application fetches it from the database and saves a copy in the Cache. When the next user requests the same data, the application bypasses the database entirely and delivers the data instantly from the Cache. This drastically reduces latency and database load.

Caches can be implemented at various levels:

1. Browser Cache: Stores images, CSS, and JS files on the user's computer so they don't have to be downloaded again.

2. CDN (Content Delivery Network): Caches static files on servers geographically closer to the user.

3. Application/Database Cache: Tools like *Redis* or *Memcached* that sit between the Application Tier and Data Tier, storing frequent database query results in memory.

🔑 Cache Concepts:

- ✓ **Cache Hit:** The requested data is found in the cache (Fast).
- ✓ **Cache Miss:** The data is not in the cache, requiring a slow fetch from the database (Slow).
- ✓ **Cache Invalidation/Eviction:** The process of removing old or updated data from the cache so users don't see outdated information.
- ✓ **TTL (Time To Live):** A setting that dictates how long data stays in the cache before it is automatically deleted.

📊 How Caching Works

```
                    USER REQUEST
                         |
                         ▼
[ CACHE CHECK ] ──(Cache Hit)──▶ [ RETURN DATA FAST ]
                         |
                    (Cache Miss)
                         |
                         ▼
[ QUERY DATABASE ] ──▶ [ STORE IN CACHE ] ──▶ [ RETURN DATA ]
```

✅ Advantages:

✓ Drastically reduces page load times and network latency.

✓ Reduces the load on backend databases, preventing crashes.

✓ Improves overall user experience and SEO rankings.

❌ Disadvantages:

✗ **Stale Data:** If the database updates but the cache isn't cleared, users will see old data.

✗ Memory (RAM) used for caching is expensive compared to hard drive storage.

Hinglish Explanation:
Cache ek temporary, super-fast memory (RAM) hoti hai. Jab bahut saare log ek hi cheez (jaise viral video ka naam) search karte hain, toh har baar database se data nikalna slow hota hai. Isliye pehli baar mein hi data ko Cache mein save kar lete hain. Agli baar jab koi wahi data mangta hai, toh database tak jane ki zaroorat nahi padti, Cache turant answer de deta hai. Isse website bahut fast ho jati hai.

5 Building Blocks: Proxies and Indexes

Definition:

A **Proxy** is an intermediary server that sits between a client and a backend server, routing requests and responses. An **Index** is a data structure technique used by databases to quickly locate and access the data without having to search every row in a database table.

Proxies (Forward vs Reverse):

Proxies play a huge role in scaling and securing web architecture. There are two main types:
1. Forward Proxy: Sits in front of *clients*. If employees in a corporate network try to access Facebook, the forward proxy intercepts the request. It can block the request (filtering), or hide the client's IP address from the internet (anonymity).
2. Reverse Proxy: Sits in front of *web servers*. When clients send requests to a website, the reverse proxy intercepts them. It protects the actual web servers from direct internet exposure, handles SSL encryption, blocks DDoS attacks, and routes traffic to the appropriate backend server. Nginx and HAProxy are popular reverse proxies.

Database Indexes:

As databases grow to millions of rows, finding a specific record (e.g., searching for a user by email) becomes extremely slow if the database has to scan every single row one-by-one (a Full Table Scan).
An **Index** works exactly like the index at the back of a textbook. Instead of reading the whole book to find a keyword, you look at the index, which tells you the exact page number. In databases, an index (usually

implemented as a B-Tree or Hash Table) stores a specific column's data in a highly organized, sorted manner along with a pointer to the original row. This turns a search that would take minutes into milliseconds.

🔑 Key Concepts:

- ✓ **Reverse Proxy Security:** Hides backend server IPs, preventing direct hacker attacks.
- ✓ **Index Read vs Write tradeoff:** Indexes drastically speed up READ queries (SELECT), but they slightly slow down WRITE queries (INSERT/UPDATE/DELETE) because the index must also be updated every time data changes.
- ✓ **Primary Keys:** In relational databases, the Primary Key is automatically indexed.

📊 Reverse Proxy & Indexing Visualized

```
[ REVERSE PROXY ]
Clients ──▶ [ Reverse Proxy ] ──▶ Hidden Server 1
                              ──▶ Hidden Server 2
(Protects servers, manages SSL, routes traffic)

[ DATABASE INDEX ]
Query: "Find user ID 50"
Without Index: Scans Row 1, Row 2, Row 3... up to 50 (Slow)
With Index: Looks up B-Tree, jumps directly to Row 50's address (Instant)
```

🌍 Applications:

- → **Proxies:** Cloudflare acts as a massive global reverse proxy to protect websites from DDoS attacks and cache content.
- → **Indexes:** E-commerce product searches utilize extensive indexing to allow users to instantly filter products by brand or price.

👉 ***Hinglish Explanation:***
Proxy *ek middleman hai. Reverse Proxy website ke servers ke aage khada hota hai bodyguard ki tarah, taaki koi hacker direct server tak na pahunch sake.* ***Index*** *database ki ek dictionary hai. Bina index ke, database ko koi naam dhoondhne ke liye lakhon rows check karni padti hain. Index banne ke baad, database direct us location par jump karta hai jahan data rakha hai, jisse search super fast ho jati hai.*

6 Building Blocks: Load Balancers and Queues

📌 Definition:

A **Load Balancer** efficiently distributes incoming network traffic across a group of backend servers. A **Message Queue** is an asynchronous communication system used to temporarily store tasks or messages

so they can be processed by backend workers in the background.

Load Balancers:

When a website becomes highly popular, a single web server (no matter how powerful) cannot handle millions of requests simultaneously. The solution is **Horizontal Scaling**—adding more servers. But how do users know which server to connect to? This is the job of a **Load Balancer**.

The Load Balancer acts as a traffic cop. It sits in front of your server farm and receives all incoming requests. It then routes each request to a specific server based on an algorithm. If a server crashes, the load balancer automatically detects the failure (via Health Checks) and stops sending traffic to the dead server, ensuring High Availability. Common routing algorithms include *Round Robin* (taking turns sequentially) and *Least Connections* (sending traffic to the server currently doing the least work).

Message Queues:

Web applications often need to perform heavy tasks, like generating a PDF report, sending 10,000 emails, or processing an uploaded video. If the web server does this immediately while the user waits, the web page will freeze and timeout.

Message Queues (like RabbitMQ, Apache Kafka, or AWS SQS) solve this by decoupling the task from the web request. When a user uploads a video, the web server quickly sends a "process video" message into the Queue and immediately responds to the user saying "Video is processing." Separately, a cluster of background "Worker Servers" continuously pulls tasks from the Queue one by one and processes them at their own pace without blocking the main website.

Key Concepts:

- ✓ **Health Checks:** Load balancers constantly ping servers to ensure they are alive before sending traffic.
- ✓ **Session Persistence (Sticky Sessions):** A load balancer setting ensuring a specific user is always routed to the same server if session data is stored locally.
- ✓ **Asynchronous Processing:** Queues enable tasks to happen in the background (asynchronously), keeping the user interface fast and responsive.
- ✓ **Decoupling:** Queues separate the web servers from the worker servers, meaning if the workers crash, the web server still functions (it just adds tasks to the queue to be processed later).

Load Balancer & Queue Architecture

```
                    [ LOAD BALANCER ]
Users ──▶ [ Load Balancer ] ──▶ Web Server 1
                                ──▶ Web Server 2
                                ──▶ Web Server 3

                    [ MESSAGE QUEUE ]
Web Server ──(Sends Task)──▶ [ QUEUE [Task 3][Task 2][Task 1] ]
                                          │
                                (Pulls Task) ──▼
                        [ Worker Server ] (Processes heavy task)
```

Real-Life Example:

Food Delivery App (Zomato/Swiggy): Millions of people open the app at dinner time. A **Load Balancer** distributes these users across 100 different servers so the app doesn't crash. When you place

an order, the system needs to send an email, a push notification, and SMS to the restaurant. Instead of doing this immediately and making you wait 10 seconds, the server pushes an "Order Placed" message into a **Queue** and instantly shows you "Order Confirmed!" Background servers read the queue and send the emails/SMS seconds later.

Hinglish Explanation:

Load Balancer *ek traffic police wala hai. Jab website par bheed aati hai, to wo aane wale logon ko alag-alag servers par bhej deta hai taaki ek server pe overload na ho.*

Queue *(Line) heavy tasks ko background mein handle karne ke liye hoti hai. Agar koi bada kaam (jaise video compress karna) hai, to server usko queue mein daal deta hai aur user ko free kar deta hai. Piche background servers line mein lage hue kaamo ko aaram se complete karte rehte hain, bina website ko slow kiye.*

7 Web Application Architecture (WAA)

Definition:

Web Application Architecture (WAA) encompasses the specific frameworks, design patterns, and structural components used to build dynamic web applications. It dictates how the client-side (frontend) and server-side (backend) interact to deliver a seamless user experience over HTTP.

Detailed Explanation:

While "Web Systems Architecture" deals with the macro-level hardware/network layout (load balancers, databases, scaling), "Web Application Architecture" deals with the software design pattern inside the application itself. It defines how code is organized to process requests and render web pages.

The foundation of WAA relies on the **Client-Server model** interacting via the stateless HTTP protocol. Because HTTP forgets who you are after every request, WAA introduces mechanisms like Sessions, Cookies, and JWTs (JSON Web Tokens) to maintain user state.

Modern WAA largely relies on the **MVC (Model-View-Controller)** design pattern:

- **Model:** Represents the data and business logic. It handles database interactions.
- **View:** What the user sees (UI). It renders HTML/CSS based on data from the Model.
- **Controller:** The middleman. It receives the HTTP request from the user, asks the Model for data, and passes that data to the View to be displayed.

Evolution of WAA: Historically, web apps used *Server-Side Rendering (SSR)*, where the server generated a full HTML page for every click. Today, modern WAA heavily features *Single Page Applications (SPAs)* using frameworks like React or Angular. In an SPA, the server only sends raw JSON data via an API (Application Programming Interface), and the client's browser uses JavaScript to dynamically build and update the UI without reloading the page.

🔍 Key Components in Modern WAA:

✓ **Frontend (Client-side):** HTML, CSS, JavaScript (React, Vue, Angular). Responsible for UX/UI.

✓ **Backend (Server-side):** Node.js, Python (Django), Java (Spring). Contains the Controller and Model logic.

✓ **RESTful APIs / GraphQL:** The communication bridge. The frontend asks the backend for data in JSON format via APIs.

✓ **Microservices:** Breaking the backend into smaller, independent mini-applications rather than one massive Monolith.

📊 Modern SPA Web Application Architecture

```
     [ CLIENT BROWSER (SPA) ]               [ BACKEND SERVER ]
+----------------------------+         +-----------------------+
|  UI Components (React/Vue) |         |      Controller       |
|                            | --API-> |          |            |
|   State Management         | <-JSON- |        Model          |
|   (Routing, Data Fetch)    |         |          |            |
+----------------------------+         +----------|------------+
                                                   ▼
                                            [ DATABASE ]
```

✅ Advantages of Modern WAA (SPAs via APIs):

✓ **Fast User Experience:** No page reloading; feels like a native desktop app.

✓ **Reusability:** The same backend API can serve the Web App, iOS App, and Android App simultaneously.

❌ Disadvantages:

✗ **SEO Challenges:** Search engine bots sometimes struggle to read dynamically generated JavaScript pages.

✗ **Complexity:** Managing frontend state and backend APIs requires advanced development skills.

👉 ***Hinglish Explanation:***
*Web Application Architecture batata hai ki software ka code kaise likha jayega. Puraane zamane mein har click par server ek naya HTML page banakar bhejta tha. Aaj kal **SPA (Single Page Application)** chalti hain, jahan frontend (jaise React) sirf raw data (JSON) mangta hai backend API se. Backend us request ko process karta hai **MVC (Model-View-Controller)** pattern ke through. Isse website bina reload huye turant update ho jati hai.*

🔁 QUICK REVISION NOTES

1. 2-Tier vs 3-Tier Architecture:
2-Tier has Client directly connecting to Database (fast but unscalable/insecure). 3-Tier adds an Application Server in the middle (Client -> App -> DB), isolating logic, improving security, and enabling scalability.

2. Caches:
High-speed RAM storage for frequently accessed data to bypass slow database queries. Radically improves speed but requires handling stale data (eviction).

3. Proxies & Indexes:
Reverse Proxies protect servers, handle SSL, and route traffic. Database Indexes act like a book index (B-Tree), allowing lightning-fast data retrieval instead of scanning every row.

4. Load Balancers & Queues:
Load Balancers distribute user traffic across multiple servers to prevent crashing. Queues (Message Brokers) handle heavy background tasks asynchronously so the main website stays fast and responsive.

5. Web Application Architecture (WAA) & MVC:
Software design pattern. MVC divides code into Model (data/DB), View (UI), and Controller (logic router). Modern apps use APIs and SPAs (Single Page Applications) for dynamic, reload-free experiences.

IMPORTANT QUESTIONS (6 Marks Each)

Q1. Differentiate between 2-Tier and 3-Tier architecture in web systems. What are the major drawbacks of 2-tier architecture that 3-tier overcomes? 6 Marks

Expected Answer Points:

- Define 2-Tier (Client + Database). Explain "Fat Client".
- Define 3-Tier (Presentation + Application + Database). Explain "Thin Client".
- Drawbacks of 2-Tier: Security (DB exposed), Poor scalability (connection limits), Maintenance difficulty (deploying logic to all clients).
- How 3-Tier fixes it: Application tier hides DB, scales independently, and centralizes business logic.

Q2. What is a Cache in web systems architecture? How does it contribute to building fast and scalable data access? 6 Marks

Expected Answer Points:

- Definition of Cache (in-memory, high-speed storage).
- Problem it solves: Database disk reads are slow; repetitive identical queries overload DB.
- Mechanism: Cache hit vs Cache miss. Data fetched once from DB, served multiple times from

RAM.

- Mention levels of caching: Browser, CDN, Application cache (Redis/Memcached).

Q3. Explain the roles of Load Balancers and Message Queues in achieving high scalability in web applications. 6 Marks

Expected Answer Points:

- Load Balancer: Distributes incoming traffic across multiple servers (Horizontal scaling). Mentions algorithms like Round Robin.
- Ensures high availability through Health Checks (rerouting traffic from dead servers).
- Message Queue: Decouples heavy tasks (e.g., sending emails) from the main request thread.
- Ensures asynchronous processing so the UI remains fast while background workers process tasks.

Q4. Discuss the concept of Database Indexes and Reverse Proxies. Why are they essential for enterprise-level web architecture? 6 Marks

Expected Answer Points:

- Reverse Proxy: Middleman sitting in front of web servers. Protects IP, handles SSL, blocks attacks.
- Database Index: Data structure (B-Tree) improving data retrieval speed.
- Explain without index: Full table scan (slow). With index: Pointer lookup (fast).
- Mention the tradeoff: Indexes speed up READs but slow down WRITEs.

Q5. What is Web Application Architecture (WAA)? Explain the MVC design pattern used in modern web applications. 6 Marks

Expected Answer Points:

- Define WAA: Structural software design of how frontend and backend interact via HTTP.
- Explain MVC: Model (Data/Logic), View (UI/HTML), Controller (Middleman/Router).
- Flow: User sends request -> Controller handles it -> Controller gets data from Model -> Controller gives data to View -> View displays it to user.
- Mention the shift towards SPAs (Single Page Applications) communicating with REST APIs using JSON.

Q6. How does a 3-Tier architecture enforce security and separation of concerns? Provide a real-world example to illustrate its layers. 6 Marks

Expected Answer Points:

- Security: DB is on a private network, accessible only by the Application server, not the internet.

- Separation of concerns: UI developers can work on frontend without touching database schema; logic is centralized.
- Example: E-commerce site. Presentation (React frontend), Application (Node.js API handling cart/payments), Data (MySQL DB storing user/product info).

📚 Unit 2 notes are comprehensive and exam-ready 📚

Master Web Systems Architecture to secure top marks in your exams!

Web Technologies - Unit 2 Notes

📚 Source: Rgpvnotes.tech | Premium Academic Content for Students

© 2026 - Your Academic Excellence Partner

These notes are created for educational purposes to help students understand complex concepts in a structured manner.